

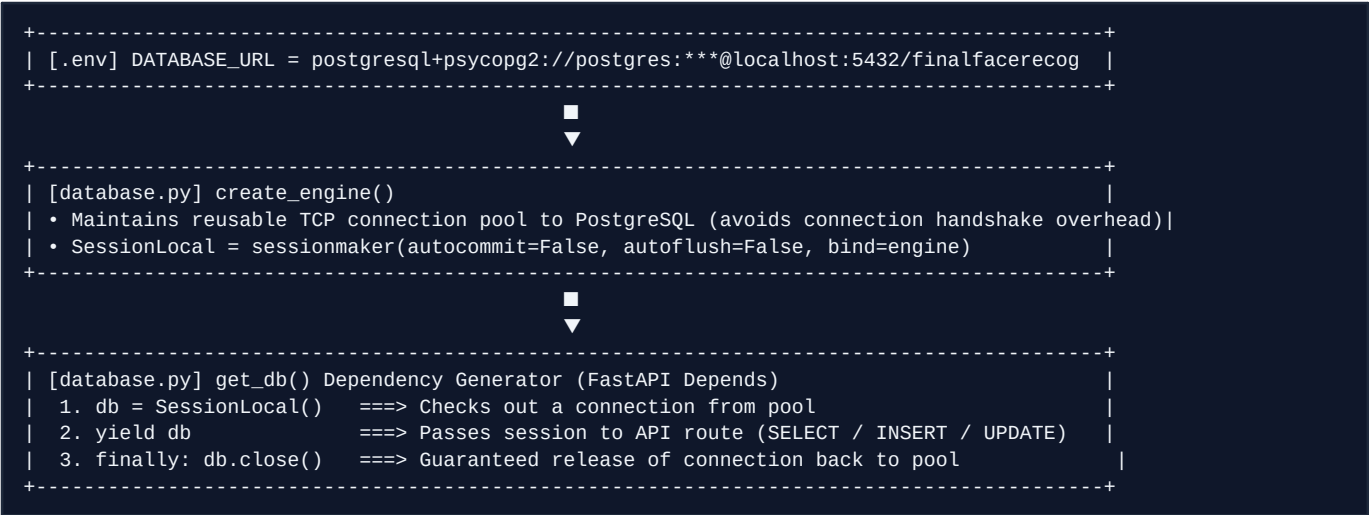
Database Connection & Architecture Guide

Deep-Dive Technical Walkthrough: PostgreSQL, SQLAlchemy Engine, Session Pool & Code

Database: PostgreSQL 15+ (finalfacerecog) | **ORM Layer:** SQLAlchemy 2.0+ | **Driver:** psycopg2-binary | **Web Framework:** FastAPI (Python)

1. High-Level Connection Architecture

The system relies on an asynchronous, dependency-injected connection pool that guarantees zero connection leaks. Below is the lifecycle of every database interaction from startup to HTTP request completion:



2. Environment Configuration (backend/.env)

The application reads credentials securely from the environment file. The PostgreSQL connection string follows standard RFC-1738 URI format:

```
# backend/.env
DATABASE_URL=postgresql+psycopg2://postgres:Biswajit2005@localhost:5432/finalfacerecog
THRESHOLD=0.6
SECRET_KEY=3e31f4f1e6414c2cadef7ae25b32b3a007e11657d1cac48d318922319423e39c
```

URI Component	Value in Project	Technical Purpose
Dialect + Driver	postgresql+psycopg2	Tells SQLAlchemy to communicate with PostgreSQL using the binary C-driver psycopg2.
User : Password	postgres : Biswajit2005	Authentication credentials for the local PostgreSQL database server.
Host : Port	localhost : 5432	Network destination address and standard PostgreSQL listening port.
Database Name	finalfacerecog	Target database holding students, attendance_records, and auth tables.

3. Connection Engine & Session Factory (backend/database.py)

Here is the complete source code of backend/database.py which defines the engine and generator:

```
# backend/database.py
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from dotenv import load_dotenv

load_dotenv()

# 1. Retrieve connection string with fallback
DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL or DATABASE_URL.startswith("DATABASE_URL="):
    DATABASE_URL = "postgresql+psycopg2://postgres:Biswajit2005@localhost:5432/finalfacerecog"

# 2. Create the Database Engine (connection pool)
engine = create_engine(DATABASE_URL)

# 3. Session factory: autocommit=False ensures transactions are explicit
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# 4. Declarative Base class for ORM models
Base = declarative_base()

# 5. FastAPI Dependency Generator
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Why the `get_db()` Generator with `yield` & `finally` is Critical:

- **Resource Isolation:** Each incoming HTTP request receives its own isolated database transaction session.
- **Automatic Garbage Collection:** When the endpoint returns a response (or throws an uncaught error), execution jumps into the `finally` block, executing `db.close()`.
- **Prevents Connection Exhaustion:** Without `finally: db.close()`, abandoned database connections stay open until PostgreSQL hits `max_connections` and crashes.

4. Automatic Table Creation on Startup (backend/app.py)

When the FastAPI application initializes, it binds the model metadata to the engine and issues a single DDL command to PostgreSQL:

```
# backend/app.py
from database import engine, Base
import models # Must import models so Base registers all tables

# Automatically creates any missing tables in PostgreSQL without overwriting existing data
Base.metadata.create_all(bind=engine)
logger.info("PostgreSQL tables ensured (create_all)")
```

5. Hybrid SQL + JSONB Table Models (backend/models.py)

Tables inherit from `Base`. Notice how PostgreSQL's native JSONB dialect is used to store dynamic arrays (AI embeddings and student attendance lists) without needing complex join tables:

```
# backend/models.py
from sqlalchemy import Column, Integer, String, Float, Boolean, DateTime
from sqlalchemy.dialects.postgresql import JSONB
from database import Base

class Student(Base):
    __tablename__ = "students"

    id = Column(Integer, primary_key=True)
    student_id = Column(String(100), unique=True, nullable=False)
    student_name = Column(String(200), nullable=False)
    department = Column(String(100))
    year = Column(String(20))
    division = Column(String(20))
    email = Column(String(255), unique=True, nullable=True)
    face_registered = Column(Boolean, default=False)
    embeddings = Column(JSONB) # Stores 512-dim facial vector as dynamic JSON array

class AttendanceRecord(Base):
    __tablename__ = "attendance_records"

    id = Column(Integer, primary_key=True)
    date = Column(String(20))
    subject = Column(String(100))
    department = Column(String(100))
    year = Column(String(20))
    division = Column(String(20))
    finalized = Column(Boolean, default=False)
    students = Column(JSONB, default=list) # Embedded array: [{"studentId": "...", "status": "present"}]
```

6. How Endpoints Execute Queries via Depends(get_db)

FastAPI routes inject the session via db: Session = Depends(get_db). Here are two real examples from the codebase:

Example A: SELECT Query (backend/student/view_attendance.py)

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from database import get_db
from models import AttendanceRecord

@attendance_router.get('/api/attendance')
def get_attendance(student_id: str, db: Session = Depends(get_db)):
    # Query executed on PostgreSQL through the active pool connection
    records = db.query(AttendanceRecord).filter_by(
        department="Computer Science",
        year="TE",
        division="A"
    ).all()

    # Session automatically closed by get_db() when return fires
    return {"success": True, "attendance": [r.to_dict() for r in records]}
```

Example B: INSERT & COMMIT Transaction (backend/auth/routes.py)

```
@auth_router.post('/api/signup')
async def api_signup(request: Request, db: Session = Depends(get_db)):
    data = await request.json()

    # 1. SELECT query to verify uniqueness
    existing = db.query(Student).filter_by(email=data['email']).first()
    if existing:
        return {"success": False, "error": "Email already registered"}

    # 2. Instantiate new model
    new_student = Student(
        student_id=data['studentId'],
        student_name=data['studentName'],
        email=data['email']
    )

    # 3. Add to transaction and commit to PostgreSQL disk
    db.add(new_student)
    db.commit()          # Explicit transaction commit
    db.refresh(new_student) # Reload generated primary key (id)

    return {"success": True, "user": new_student.to_dict()}
```

7. Summary: Core Database Guarantees

1. **Pooled Connections:** create_engine maintains a pool of persistent connections, avoiding per-request TCP handshakes.
2. **Safe Dependency Lifecycle:** Depends(get_db) uses Python generators to lease a connection on route entry and release it on exit.
3. **ACID Transactions:** autocommit=False forces explicit db.commit(), ensuring partial writes never corrupt database state.
4. **JSONB Extensibility:** Dynamic AI facial vectors and nested attendance lists are stored natively without performance penalties.